



Affine Sessions

Dimitris Mostrous, Vasco Thudichum Vasconcelos

► To cite this version:

Dimitris Mostrous, Vasco Thudichum Vasconcelos. Affine Sessions. 16th International Conference on Coordination Models and Languages (COORDINATION), Jun 2014, Berlin, Germany. pp.115-130, 10.1007/978-3-662-43376-8_8 . hal-01290071

HAL Id: hal-01290071

<https://inria.hal.science/hal-01290071>

Submitted on 17 Mar 2016

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons Attribution 4.0 International License

Affine Sessions

Dimitris Mostrous and Vasco Thudichum Vasconcelos

University of Lisbon, Faculty of Sciences and LaSIGE
Lisbon, Portugal

Abstract. Session types describe the structure of protocols from the point of view of each participating channel. In particular, the types describe the type of communicated values, and also the dynamic alternation of input and output actions on the same channel, by which a protocol can be statically verified. Crucial to any term language with session types is the notion of linearity, which guarantees that channels exhibit exactly the behaviour prescribed by their type. We relax the condition of linearity to that of affinity, by which channels exhibit at most the behaviour prescribed by their types. This more liberal setting allows us to incorporate an elegant error handling mechanism which simplifies and improves related works on exceptions. Moreover, our treatment does not affect the progress properties of the language: sessions never get stuck.

1 Introduction

A session is “*a semantically atomic chain of communication actions which can interleave with other such chains freely, for high-level abstraction of interaction-based computing*” [21]. *Session types* capture this intuition as a description of the structure of a protocol, in the simplest case between two programs (binary sessions). This description consists of types that indicate whether a communication channel will next perform an output or input action, the type of the value to send or receive, and what to do next, inductively. For example, $!nat.!string.?bool.end$ is the type of a channel that will first send a value of type `nat`, then one of type `string`, then wait for a value of type `bool`, and nothing more. This type can be materialised by the π -calculus [19] process $\bar{a}5.\bar{a}\text{“hello”}.a(x).\mathbf{0}$. To compose two processes that communicate over a channel, we require that each has a complementary (or *dual*) type, so that an input will match an output, and vice versa. The dual of the previous type is $?nat.?string.!bool.end$, and can be implemented by $a(x).a(y).\bar{a}(x + 1 < 2).\mathbf{0}$. To ensure that the actions take place in the prescribed order, session typing relies crucially on the notion of *linearity* [12], which means that a causal chain can be assumed. To see why, imagine that we write the first process as $\bar{a}5.\mathbf{0} \mid \bar{a}\text{“hello”}.a(x).\mathbf{0}$. Now we cannot determine which output can react first, and the second process can receive a “hello” first, which would clearly be unsound and would most likely raise an error.

Beyond the basic input/output types, sessions typically provide constructors for alternative sub-protocols, which are very useful for structured interaction. For example, $\&\{go: T_1, cancel: T_2\}$ can be assigned to an (external) choice $a \triangleright$

$\{\text{go}.P_1 \parallel \text{cancel}.P_2\}$. The dual type, where $\overline{T}$ denotes T with an alternation of all constructors, is $\oplus \{\text{go}: \overline{T_1}, \text{cancel}: \overline{T_2}\}$, and corresponds to a process that will make a (internal) choice, either $\overline{a} \triangleleft \text{go}.Q_1$ or $\overline{a} \triangleleft \text{cancel}.Q_2$. In the first case the two processes will continue as P_1 and Q_1 , respectively.

As can be seen, sessions are very suitable as a static verification mechanism for interacting programs. However, they are also quite rigid, since everything in the description of a session type *must* be implemented by a program with that type. Indeed, in many real world situations, interactions are structured but can be aborted at any time, for example an online store should be prepared for clients that get disconnected, that close their web browsers, or for general *errors* that abruptly severe the expected pattern of interaction.

In this work, we address the above issues. In technical terms, we relax the condition of linearity to that of *affinity*, inspired by Affine Logic (which is the variation of Linear Logic with unrestricted weakening; see [2] for an introduction), and this allows processes to terminate their sessions prematurely. However, a naive introduction of affinity can leave programs in a stuck state: let us re-write the first process into $\overline{a}5.\overline{a}\text{"hello"}.0$, i.e., without the final input; then, after two communications the dual process will be stuck trying to perform $\overline{a}(5 + 1 < 2).0$. One of the basic tenets of sessions, *progress*, is now lost. Actually, the study of Proof Nets for Affine Logic [2] reveals that weakening is not, and should not be, invisible. In particular, there exists a device that will perform the weakening step by step, progressing through the dependencies of a proof, and removing all that must be removed. This is exactly what we need in order to handle an abrupt termination of an interaction in an explicit way, and we denote it by $a\cancel{}$, which reads *cancel a*. Now we can write $\overline{a}5.\overline{a}\text{"hello"}.a\cancel{}$, and after two steps against the dual process we obtain $\overline{a}(5 + 1 < 2).0 \mid a\cancel{}$, which results in the cancellation of the output (and in general, of any subsequent actions on a). We take this idea a step further: if cancellation of a session is explicit, we can treat it as an *exception*, and for this we introduce a *do-catch* construct that can provide an alternative behaviour activated when a cancellation is encountered. For example, we can write $\text{do } \overline{a}(5 + 1 < 2).0 \text{ catch } P$, and a composition with $a\cancel{}$ will *replace* $\overline{a}(5 + 1 < 2).0$ with the exception handler P . A *do-catch* is not the same as the *try-catch* commonly found in sequential languages: it does not define a persistent scope that captures exceptions from the inside, but rather it applies to the first communication and is activated by exceptions from the outside (as in the previous example). Thus, $\text{do } \overline{a}(5 + 1 < 2).0 \text{ catch } P$ in parallel to $a(x).0$ becomes 0 , because the communication was successful.

2 Affine Sessions by Example

We describe a simple interaction that implements a book purchase taking place between three processes, **Buyer**, **Seller**, and **Bank**. The buyer sends the title of a book, receives the price, and makes a choice to either buy it or cancel. If the buyer chooses to buy the book, the credit card is sent over the session, and the

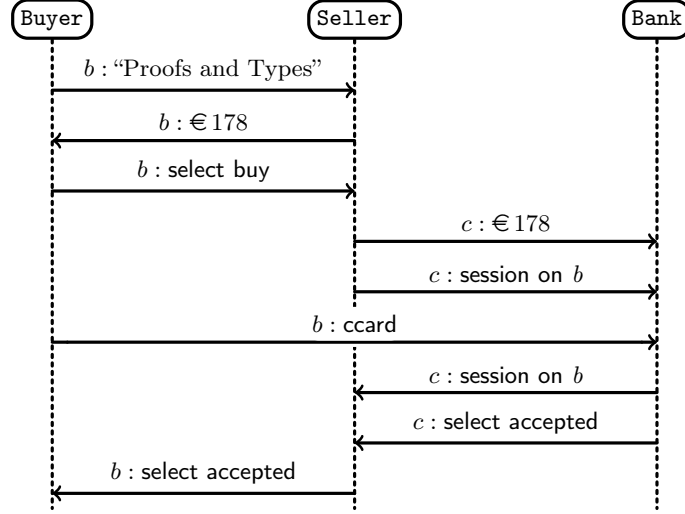


Fig. 1. Sequence Diagram for Successful Book Purchase

buyer is informed whether or not the transaction was successful. The diagram in Figure 1 shows the interactions of a specific purchase.

We now show how this scenario can be implemented using sessions, and how our treatment of affinity can be used to enable a more concise and natural handling of exceptional outcomes. Our language is an almost standard π -calculus where replication is written $\text{acc } a(x).P$ and plays the role of “accept” in sessions terminology [15]. Dually, an output that activates a replication is written $\text{req } \bar{a}b.P$, and is called a “request”. We will use some standard constructs that are encodable in π -calculus, like $\bar{a}e$ for an expression e . Also, we use $\text{if } t \text{ then } P \text{ else } Q$ that can be implemented by a new session, specifically $a \triangleright \{\text{true}.P \parallel \text{false}.Q\}$ against some process representing the test t that communicates the result by a selection of one of the labels, $\bar{a} \triangleleft \text{true}$ or $\bar{a} \triangleleft \text{false}$.

An implementation of the protocol we described in Figure 1 is shown below:

$$\begin{aligned}
 \text{Buyer} &\doteq (\nu b) \left(\text{req } \overline{\text{seller}}b \mid \bar{b} \text{“Proofs and Types”}.b(\text{price}).\text{if } (\text{price} < 200) \right. \\
 &\quad \left. \text{then } \bar{b} \triangleleft \text{buy}.\bar{b} \text{ccard}.b \triangleright \{\text{accepted}.P \parallel \text{rejected}.Q\} \text{ else } \bar{b} \triangleleft \text{cancel} \right) \\
 \text{Seller} &\doteq \text{acc } \text{seller}(s). \left(s(\text{prod}).\bar{s} \text{price}(\text{prod}). \right. \\
 &\quad \left. s \triangleright \{ \text{buy}.(\nu c)(\text{req } \overline{\text{bank}}c \mid \bar{c} \text{price}(\text{prod}).\bar{c}s.c(r). \right. \\
 &\quad \quad \left. c \triangleright \{\text{accepted}.\bar{r} \triangleleft \text{accepted} \parallel \text{rejected}.\bar{r} \triangleleft \text{rejected}\}) \right. \\
 &\quad \quad \left. \parallel \text{cancel}.0 \} \right) \\
 \text{Bank} &\doteq \text{acc } \text{bank}(k). \left(k(\text{amount}).k(r).r(\text{card}).\bar{k}r. \right. \\
 &\quad \left. \text{if } \text{charge}(\text{amount}, \text{card}) \text{ then } \bar{k} \triangleleft \text{accepted} \text{ else } \bar{k} \triangleleft \text{rejected} \right)
 \end{aligned}$$

First we note how sessions are established. For example, in **Buyer** the fresh name b is sent to **Seller** via the request $\text{req } \overline{\text{seller}}b$, where it will substitute s in a copy of the replication, and it appears also locally. These are the two endpoints of the session, and it is easy to check that the interactions match perfectly. Another point is the “borrowing” of the session b (which becomes identified with s) from **Seller** to **Bank**, with $\overline{cs}.c(r)$ and $k(r).r(\text{card}).\overline{kr}$ (again, c and k are identified), respectively, so that the credit card is received directly by **Bank**; see also Figure 1.

A more robust variation of **Seller** could utilise the do-catch mechanism to account for the possibility of the **Bank** not being available (or being crashed), by providing an alternative payment provider. This can be achieved if we substitute $\text{req } \overline{\text{bank}}c$ with $\text{do req } \overline{\text{bank}}c \text{ catch req } \overline{\text{paymate}}c$, so that a failure to use the bank service ($\text{bank}\downarrow$) will activate $\text{req } \overline{\text{paymate}}c$ (which must have the same type) and the protocol has a chance to complete successfully.

The **Buyer** might also benefit from our notion of exception handling. For example, we show an adaptation that catches a cancellation at the last communication of the **buy** branch and prints an informative message:

$$\text{BuyerMsg} \doteq (\nu b) \left(\begin{array}{l} \text{req } \overline{\text{seller}}b \mid \overline{b} \text{“Proofs and Types”}.b(\text{price}). \\ \text{if } (\text{price} < 200) \text{ then } \overline{b} \triangleleft \text{buy}.\overline{b} \text{ccard}. \\ \quad \text{do } b \triangleright \{ \text{accepted}.P \parallel \text{rejected}.Q \} \\ \quad \text{catch req } \overline{\text{print}} \text{“Error 42”} \\ \text{else } \overline{b} \triangleleft \text{cancel} \end{array} \right)$$

As mentioned in the Introduction, a do-catch on some communication does not catch subsequent cancellations. For instance, if in the above example the do-catch was placed on $\overline{b} \text{“Proofs and Types”}$, then any $b\downarrow$ generated *after* this output was performed would be uncaught, since $\text{req } \overline{\text{print}} \text{“Error 42”}$ would have been already discarded. However, a do-catch does catch cancellations emitted *before* the point of definition, so placing it near the end of a protocol is very useful if we just want a single exception handler that catches everything.

Indeed, exception handlers that persist for the lifetime of the whole session are definable. Specifically, we can write $\text{try } P \text{ catch } (b: Q)$ to mean that one endpoint of b is implemented in P and that Q should be activated if b is canceled at *any* point from the *outside* of P . This try-catch notation translates to a do-catch on the last prefix on b in P , in multiple branches as required, assuming that b is not delegated (i.e., sent over another channel), as is the case in **BuyerMsg**.¹ Then we have, for example, that $\text{try } \overline{b}5.P' \text{ catch } (b: Q) \mid b(x).0$ becomes $\text{try } P' \text{ catch } (b: Q)$ and $\text{try } \overline{b}5.P' \text{ catch } (b: Q) \mid b\downarrow$ becomes $P'' \mid Q$ where P'' is P' with b and all its dependencies canceled. In general, however, our mechanism is very fine-grained, and a single session can have multiple, nested do-catch on crucial points of communication.

¹ We assume that in **BuyerMsg** P and Q do not use b . The restriction to non-delegated sessions is for simplicity: if the last action on b was to send it over some channel k , e.g., using $\overline{kb}$, the encoding would be more complicated, because we would not have access to the end of the session and anything after that output would not be caught.

Note also that $a\cancel{z}$ can be very useful in itself, even without the do-catch mechanism. Here are two ways to implement a process that starts a protocol with Seller only to check the price of a book:

$$\begin{aligned}\text{CheckPriceA} &\doteq (\nu b) \left(\text{req } \overline{\text{seller}b} \mid \bar{b} \text{“The Prince”}.b(\text{price}).(\bar{b} \triangleleft \text{cancel} \mid R) \right) \\ \text{CheckPriceB} &\doteq (\nu b) \left(\text{req } \overline{\text{seller}b} \mid \bar{b} \text{“Beyond Good and Evil”}.b(\text{price}).(b\cancel{z} \mid R) \right)\end{aligned}$$

Both the above processes can be typed. However, the first requires a knowledge of the protocol, which in that case includes an exit point (branch **cancel**), while the second is completely transparent. For example, imagine a buyer that selects buy by accident and then wishes to cancel the purchase: without cancellation it is impossible because it is not *predicted* by the session type; with cancellation it is extremely simple, as shown below.

$$\text{BuyerCancel} \doteq (\nu b) \left(\text{req } \overline{\text{seller}b} \mid \bar{b} \text{“Gödel, Escher, Bach”}.b(\text{price}).\bar{b} \triangleleft \text{buy}.b\cancel{z} \right)$$

We now make a small digression to discuss how our affine sessions can be encoded in standard sessions. The purpose is to shed light on the complexity that is required, which motivates even more our development. First of all, it is possible that both endpoints of a session emit a cancellation, possibly at different moments. Therefore, if we are to encode this behaviour in a standard session system, we must allow a protocol to end at any point and by the request of either of the participants. This can be achieved by an exchange of a decision, to **go** or to **cancel**, by both endpoints, before all communications. We show the translation for output and input; the rest is similar.

$$\llbracket !T_1.T_2 \rrbracket \doteq \oplus \left\{ \begin{array}{l} \text{go} : \& \{ \text{go} : !T_1.\llbracket T_2 \rrbracket, \text{cancel} : \text{end} \}, \\ \text{cancel} : \& \{ \text{go} : \text{end}, \text{cancel} : \text{end} \} \end{array} \right\} \quad \begin{array}{l} \llbracket ?T_1.T_2 \rrbracket \doteq \overline{\llbracket !T_1.\overline{T_2} \rrbracket} \\ \llbracket \text{end} \rrbracket \doteq \text{end} \end{array}$$

The notable point is that by an alternation of constructors we obtain a translation that preserves duality, and it is easy to check that it preserves soundness. Moreover, the only way to proceed is if both ends agree to **go**. The term-level translation follows the structure of the types, and $a\cancel{z}$ becomes $\bar{a} \triangleleft \text{cancel}$. All free sessions in the term must be canceled in the branches that do not result in normal execution so as to obtain the same typing environment, but this is always possible.

The above translation only handles cancellation. Our do-catch mechanism can also be encoded within the branches of the previous translation, but it becomes quite complicated due to the typing constraints that must be respected. In any case, we think it is obvious that the burden is heavy if one wishes to obtain a functionality as general as the one available in the affine system, and types would become completely illegible from the multiplication of constructors.

3 The Process Calculus of Affine Sessions

Syntax Our language is a small extension of standard π -calculus [19]. With respect to standard sessions systems [11,22], we avoid the need for *polarities*

and *double binders* by carefully introducing a logically-founded typing principle, detailed later. For technical convenience we shall consider all indexing sets I to be totally ordered, so that we can speak, e.g., of the maximum element. Also for technical convenience, we separate the prefixes denoted by ρ , i.e., all communication actions except for accept (replication). We only added two non-standard constructs: the *cancellation* $a\cancel{\cdot}$ and the *do-catch* construct that captures a cancellation, denoted by $\text{do } \rho \text{ catch } P$. Notice that we restricted the action to a prefix in ρ , but this is not so limiting. In the case of replication, it does not make sense to catch an event that never occurs, since as we shall see we never explicitly cancel a persistent service. For parallel composition, it would be ambiguous to allow $\text{do } (P \mid Q) \text{ catch } R$ since more than one action can be active in $(P \mid Q)$, and moreover we do not think it would really add any benefit since we can add a separate do-catch for each session. Similarly, $\text{do } a\cancel{\cdot} \text{ catch } P$ would be strange: it would allow to trigger some behaviour when the other end is canceled, but while at the same time the protected session is canceled too. It can be added if a good use is discovered, but we preferred to keep the semantics simpler.

$\rho ::= a(x : T).P$	(input)
$\bar{a}b.P$	(output)
$a \triangleright \{l_i.P_i\}_{i \in I}$	(branching)
$\bar{a} \triangleleft l_k.P$	(selection)
$\text{req } \bar{a}b.P$	(request)
$P ::= \rho$	(prefix)
$\text{acc } a(x : T).P$	(replicated accept)
$\mathbf{0}$	(nil)
$P \mid Q$	(parallel)
$(\nu a : T)P$	(restriction)
$a\cancel{\cdot}$	(cancel)
$\text{do } \rho \text{ catch } P$	(catch)

Structural Congruence With $\equiv$ we denote the least congruence on processes that is an equivalence relation, equates processes up to α -conversion, satisfies the abelian monoid laws for parallel composition, the usual laws for scope extrusion, and satisfies the axiom:

$$(\nu a : T)(a\cancel{\cdot} \mid \cdots \mid a\cancel{\cdot}) \equiv \mathbf{0}$$

We added this axiom mainly for the left-to-right direction which allows “leftover” cancellations to disappear; this is convenient for technical reasons.

Reduction We use two kinds of contexts, $C[\cdot]$ which are standard, and $H[\cdot]$ for (possible) exception handling, defined below.

$$\begin{aligned} \text{Standard Contexts : } C[\cdot] &::= \cdot \mid (C[\cdot] \mid P) \mid (\nu a : T) C[\cdot] \\ \text{Do-Catch Contexts : } H[\cdot] &::= \cdot \mid \text{do } \cdot \text{ catch } P \end{aligned}$$

Reduction is defined in two parts, first the standard rules, and then the cancellation rules. The standard reductions are defined below:

$$\begin{aligned} H_1[\bar{a}b.P] \mid H_2[a(x : T).Q] &\longrightarrow P \mid Q\{b/x\} & (\text{R-Com}) \\ H_1[\bar{a} \triangleleft l_k.P] \mid H_2[a \triangleright \{l_i.Q_i\}_{i \in I}] &\longrightarrow P \mid Q_k \quad (k \in I) & (\text{R-Bra}) \\ H[\text{req } \bar{a}b.P] \mid \text{acc } a(x : T).Q &\longrightarrow P \mid Q\{b/x\} \mid \text{acc } a(x : T).Q & (\text{R-Ses}) \\ P \equiv P' &\longrightarrow Q' \equiv Q \Rightarrow P \longrightarrow Q & (\text{R-Str}) \\ P &\longrightarrow Q \Rightarrow C[P] \longrightarrow C[Q] & (\text{R-Ctx}) \end{aligned}$$

The only notable point is that we discard any do-catch handlers, since there is no cancellation, which explains why the H -contexts disappear.

The cancellation reductions follow:

$$\begin{aligned} \text{req } \bar{a}b.P \mid a \dot{\downarrow} &\longrightarrow a \dot{\downarrow} \mid b \dot{\downarrow} \mid P & (\text{C-Req}) \\ \bar{a}b.P \mid a \dot{\downarrow} &\longrightarrow a \dot{\downarrow} \mid b \dot{\downarrow} \mid P & (\text{C-Out}) \\ a(x : T).P \mid a \dot{\downarrow} &\longrightarrow (\nu x : T)(a \dot{\downarrow} \mid x \dot{\downarrow} \mid P) & (\text{C-Inp}) \\ \bar{a} \triangleleft l_k.P \mid a \dot{\downarrow} &\longrightarrow P \mid a \dot{\downarrow} & (\text{C-Sel}) \\ a \triangleright \{l_i.P_i\}_{i \in I} \mid a \dot{\downarrow} &\longrightarrow P_k \mid a \dot{\downarrow} \quad \max(I) = k & (\text{C-Bra}) \\ \text{do } \rho \text{ catch } P \mid a \dot{\downarrow} &\longrightarrow P \mid a \dot{\downarrow} \quad \text{subject}(\rho) = a & (\text{C-Cat}) \end{aligned}$$

We discuss some notable points.

Only what is strictly needed will be deleted, in particular one might have expected $a(x : T).P \mid a \dot{\downarrow}$ to result in the annihilation of P , which can be done by generating $b \dot{\downarrow}$ for each b in the free names of P . However, this has several drawbacks: first, it is too absolute, since some interactions in P may not depend on a or x , and we prefer to preserve them; second, it is technically simpler, since in this setting we can use typing restrictions to avoid the creation of any $b \dot{\downarrow}$ for a replication $\text{acc } b(x).Q$ inside P , which follows our decision to never delete services; finally, it is what happens in Proof Nets for Affine Logic (see [2]).

In the cancellation of branching, (C-Bra), we choose the maximum index k which exists by our assumption that index sets are totally ordered. This is a simple way to avoid non-determinism solely by cancellation.² Notice that it follows the pattern of activating a continuation, motivated above.

In the rule (C-Cat), we use a function $\text{subject}(\rho)$ which returns the subject in the prefix of ρ . This is defined in the obvious way, e.g., $\text{subject}(\bar{a}b.P) = \text{subject}(\text{req } \bar{a}b.P) = a$, and similarly for the other cases. The typing system will ensure that a does not appear in P , so it is ok to leave $a \dot{\downarrow}$ in the result; this is

² The language remains confluent, as expected in a logically founded system.

needed for canceled requests, where the $a\cancel{z}$ should remain until it reacts with all of them.

We clarify some of the main points:

- i) Consider $a(x).(Q \mid \text{acc } b(z).R) \mid a\cancel{z}$. The replication provided on b may or may not depend on x . A cancellation of a does not necessarily mean that b will be affected, but if x appears in R it is possible that subsequent sessions will be canceled.
- ii) Consider $a(x).(\text{acc } x(z).Q \mid \bar{b}x.R) \mid a\cancel{z}$. As can be seen, the replicated channel x is delegated on b , and it should not be deleted just because a is canceled. Indeed, this situation is not allowed by the restrictions in our type system. In other words, some sessions *cannot* be canceled.
- iii) Consider $a(x).(Q \mid \bar{b}e.R) \mid a\cancel{z}$. The session output $\bar{b}e.R$ will not be canceled, but since it is possible that $e = x$ and in general e could appear in R , other cancellations may eventually be generated.
- iv) A communication discards any handlers: $\text{do } \bar{a}b.P \text{ catch } Q \mid a(x:T).R \longrightarrow P \mid R\{b/x\}$. The type system ensures that it is sound to discard Q , since it contains the same sessions as $\bar{a}b.P$, except for a .
- v) A cancellation activates a handler, which may provide some default values to a session, completing it or eventually re-throwing a cancellation, as in: $\text{do } \bar{a}b.P \text{ catch } (\bar{b}5.b\cancel{z}) \mid a\cancel{z} \longrightarrow \bar{b}5.b\cancel{z} \mid a\cancel{z}$.

4 Typing Affine Sessions

Types The session types we use are standard [15] with two exceptions. First, following [22] we allow a session type to evolve into a shared type. Second, we decompose shared types into accept types $\text{acc } T$ and request types $\text{req } T$, following the logical principles of Affine Logic. Technically, $\text{acc } T$ corresponds to $!T$ (“of course T ”) and $\text{req } T$ to $?T$ (“why not T ”) [12]. This has several technical advantages that simplify our development, for example $\text{acc } T$ retains information on the persistence of a term with that type, since it must be replicated, which is useful for typing. Moreover, $\text{req } T$ is the only type allowed in the context of a resource that can be used zero or more times.

$T ::= \text{end}$	(nothing)
$\mid !T.T$	(output)
$\mid ?T.T$	(input)
$\mid \oplus \{l_i : T_i\}_{i \in I}$	(selection)
$\mid \& \{l_i : T_i\}_{i \in I}$	(branching)
$\mid \text{req } T$	(request)
$\mid \text{acc } T$	(accept)

Duality The two ends of a session are composed when their types are *dual*, which is defined as an involution over the type constructors, similarly to Linear Logic’s

negation except that **end** is self-dual.³

$$\begin{aligned}
\overline{!T_1.T_2} &\doteq ?T_1.\overline{T_2} & \overline{?T_1.T_2} &\doteq !T_1.\overline{T_2} \\
\overline{\oplus \{l_i : T_i\}_{i \in I}} &\doteq \& \{l_i : \overline{T_i}\}_{i \in I} & \overline{\& \{l_i : T_i\}_{i \in I}} &\doteq \oplus \{l_i : \overline{T_i}\}_{i \in I} \\
\overline{\text{req } T} &\doteq \text{acc } T & \overline{\text{acc } T} &\doteq \text{req } T & \overline{\text{end}} &\doteq \text{end}
\end{aligned}$$

Typing Rules Typing judgements take the form:

$$P \triangleright \Gamma \quad \text{where} \quad \Gamma ::= \emptyset \mid \Gamma, a : T$$

meaning that term P has interface Γ . We shall also use Γ, Δ, Θ for interfaces.

We restrict replications to be unique, but allow multiple requests to take place against them. This means that processes can have multiple uses of $a : \text{req } T$, which corresponds to the logical principle of contraction. For this, we make use of the splitting relation from [22]:

$$\begin{aligned}
\overline{\emptyset} &= \emptyset \circ \emptyset & \overline{\Gamma, a : \text{req } T} &= (\Gamma_1, a : \text{req } T) \circ (\Gamma_2, a : \text{req } T) \\
&& \Gamma &= \Gamma_1 \circ \Gamma_2 \\
\overline{\Gamma, a : T} &= (\Gamma_1, a : T) \circ \Gamma_2 & \overline{\Gamma, a : T} &= \Gamma_1 \circ (\Gamma_2, a : T) \\
&& \Gamma &= \Gamma_1 \circ \Gamma_2
\end{aligned}$$

In the typing rules, $\text{req } \Gamma$ stands for an interface of the shape $a_1 : \text{req } T_1, \dots, a_n : \text{req } T_n$. Similarly, $\text{end } \Gamma$ stands for an interface $a_1 : \text{end}, \dots, a_n : \text{end}$.

We also define a predicate $\text{no-requests}(T)$, used to forbid any request type from appearing in the type of $a_i^\perp$, since this maps by duality to the deletion of a persistent accept on the other side, which we do not allow.⁴

$$\begin{aligned}
\text{no-requests}(\text{req } T) &= \text{false} \\
\text{no-requests}(\text{acc } T) &= \text{no-requests}(T) & \text{no-requests}(\text{end}) &= \text{true} \\
\text{no-requests}(!T_1.T_2) &= \text{no-requests}(\overline{T_1}) \wedge \text{no-requests}(T_2) \\
\text{no-requests}(?T_1.T_2) &= \text{no-requests}(T_1) \wedge \text{no-requests}(T_2) \\
\text{no-requests}(\oplus \{l_i : S_i\}_{i \in I}) &= \text{no-requests}(\& \{l_i : S_i\}_{i \in I}) = \bigwedge_{i \in I} \text{no-requests}(S_i)
\end{aligned}$$

The typing rules are presented in Figure 2. We focus on some key points. First, we type modulo structural equivalence, a possibility suggested by [18] and used in [4]. This is because associativity of “ $\mid$ ” does not preserve typability, i.e., a composition between P and $(Q \mid R)$ may be untypable as $(P \mid Q) \mid R$; see

³ The expert might notice that logical negation suggests a dualisation of all components, e.g., $\overline{!T.T'} \doteq ?\overline{T}.\overline{T'}$. In fact the output type $!T.T'$ and the request $\text{req } T$ hide a duality on T , effected by the type system, so everything is compatible.

⁴ This method works fine until one adds a second-order fragment: then type substitutions must be carefully controlled, or some results will become slightly weaker.

$$\begin{array}{c}
\text{(Out)} \quad \frac{P \triangleright \Gamma, a: T_2}{\bar{a}b.P \triangleright (\Gamma, a: !T_1.T_2) \circ b: T_1} \quad \text{(In)} \quad \frac{P \triangleright \Gamma, x: T_1, a: T_2}{a(x: T_1).P \triangleright \Gamma, a: ?T_1.T_2} \\
\\
\text{(Sel)} \quad \frac{P \triangleright \Gamma, a: T_k \quad k \in I}{\bar{a} \triangleleft l_k.P \triangleright \Gamma, a: \oplus \{l_i: T_i\}_{i \in I}} \quad \text{(Bra)} \quad \frac{\forall i \in I. P_i \triangleright \Gamma, a: T_i \quad I \neq \emptyset}{a \triangleright \{l_i.P_i\}_{i \in I} \triangleright \Gamma, a: \& \{l_i: T_i\}_{i \in I}} \\
\\
\text{(Req)} \quad \frac{P \triangleright \Gamma}{\text{req } \bar{a}b.P \triangleright \Gamma \circ a: \text{req } T \circ b: T} \quad \text{(Acc)} \quad \frac{P \triangleright \text{req } \Gamma, x: T}{\text{acc } a(x: T).P \triangleright \text{req } \Gamma, a: \text{acc } T} \\
\\
\text{(Par)} \quad \frac{P \triangleright \Gamma_1 \quad Q \triangleright \Gamma_2}{P \mid Q \triangleright \Gamma_1 \circ \Gamma_2} \quad \text{(ParSes)} \quad \frac{P \triangleright \Gamma_1, a: \text{acc } T \quad Q \triangleright \Gamma_2, a: \text{req } T}{P \mid Q \triangleright (\Gamma_1 \circ \Gamma_2), a: \text{acc } T} \\
\\
\text{(Res)} \quad \frac{P \triangleright \Gamma_1, a: T \quad Q \triangleright \Gamma_2, a: \bar{T}}{(\nu a: T)(P \mid Q) \triangleright \Gamma_1 \circ \Gamma_2} \quad \text{(Str)} \quad \frac{Q \triangleright \Gamma \quad Q \equiv P}{P \triangleright \Gamma} \quad \text{(Nil)} \quad \frac{}{\mathbf{0} \triangleright \text{req } \Gamma, \text{end } \Delta} \\
\\
\text{(Catch)} \quad \frac{\rho \triangleright \Gamma, a: T \quad P \triangleright \Gamma \quad \text{subject}(\rho) = a}{\text{do } \rho \text{ catch } P \triangleright \Gamma, a: T} \quad \text{(Cancel)} \quad \frac{\text{no-requests}(T)}{a \cancel{\downarrow} \triangleright a: T}
\end{array}$$

Fig. 2. Affine Session Typing

(ParSes), (Res). This applies also to (νa) that causes similar problems. In fact, the splitting of the terms in (Res) is inspired by the work [4] which interprets sessions as propositions in a form of Intuitionistic Linear Logic. It is because of this separation of terms, which applies also to (ParSes), that we can avoid channel polarities: the two ends of a session can never become causally dependent or intermixed. The purpose of (ParSes) is to type multiple requests against a persistent accept, which explains why $a: \text{acc } T$ remains in the conclusion. An output $\bar{a}b.P$ records a conclusion $b: T_1$, so in fact it will compose against $b: \bar{T}_1$. Therefore $!T_1.T_2$ really means “send $\bar{T}_1$,” which matches with the dual input. A cancellation $a \cancel{\downarrow}$ can be given any type that does not contain a request, as explained previously.

A do-catch is typed as follows: if ρ is an action on a and has an interface $\Gamma, a: T$, then the handler P will implement Γ , i.e., all sessions of ρ *except* for $a: T$ which has been canceled. Of course, inside P these other sessions can be canceled anyway, which corresponds to “re-throwing” the cancellation, but they can also be implemented in whole or in part. The rule is sound, since no session is damaged, irrespectively of which term we execute, ρ or P .

Motivating the “no requests” restriction on the type of $a \cancel{\downarrow}$ There are pragmatic motivations behind our decision to not allow cancellation of replicated terms,

namely that we do not wish a request to cancel a service possibly shared by many processes. However, there are also technical challenges, stemming from the fact that multiple actions of type a : $\text{req } T$ can appear in a well-typed term, which as we explain below can create ambiguity in cancellation reductions.

Let us assume that the $\text{no-requests}(T)$ restriction was lifted. Now, as an example consider the composition $\overline{\text{req } \bar{a}b.P \mid a\cancel{z} \mid \bar{c}a.Q \mid c(x).\text{acc } x(y).R}$. First, let us look at the underlined term: it is impossible to know what is the type of $a\cancel{z}$, as it could be either $\text{acc } T$ or $\text{req } T$. If the type is $\text{acc } T$, which means that the (dual) accept is canceled, we should apply cancellation to $\text{req } \bar{a}b.P$; if the type is $\text{req } T$, i.e., if $a\cancel{z}$ is in fact the cancellation of another request, then we should not touch $\text{req } \bar{a}b.P$. In our example, it is easy to check that the replication will appear after a communication on c so the type of $a\cancel{z}$ must be $\text{req } T$, but in general it is not possible to determine this information (again, consider just the underlined term). Our restriction on the type of $a\cancel{z}$ ensures that, in a case like the underlined term above, we can be sure that the type cannot be $\text{req } T$, so it must be $\text{acc } T$ and we can proceed to cancel $\text{req } \bar{a}b.P$ using (C-Req). Indeed, the full composition is not typable since in that case the type of $a\cancel{z}$ must be $\text{req } T$, and $\text{no-requests}(\text{req } T)$ is not true. In fact, without our restriction, (C-Req) does not work any more, since it assumes $a\cancel{z}$ to be of type $\text{acc } T$, so we would need to replace it with:

$$(\nu a : T)(\prod_{i \in I} \text{req } \bar{a}b_i.P_i \mid a\cancel{z}) \longrightarrow (\nu a : T)(\prod_{i \in I} (b_i\cancel{z} \mid P_i) \mid a\cancel{z}) \quad (\text{C-Req}')$$

This is the special case in which we know the type of $a\cancel{z}$ must be $\text{acc } T$, since it is bound and all other elements are requests. This variation is more complex, and we would also have to forego the ability to use do-catch on $\text{req } \bar{a}b.P$, so we chose not to introduce it. Even if we did use this seemingly more liberal system, we would anyway not want replications to be deleted, so it would be of limited value. The only advantage of this alternative solution is that it does not put restrictions on the shape of types assigned to $a\cancel{z}$, and therefore it works also with polymorphism (i.e., a second-order setting).

Typing the Book Purchase Example

It is easy to verify that the examples from Section 2 are well-typed. For the Buyer we obtain the following (for some $\text{req } \Gamma_1$ and $\text{end } \Delta_1$):

$$\text{Buyer} \triangleright \text{req } \Gamma_1, \text{end } \Delta_1, \text{seller: req } T_1$$

$$\text{with } \overline{T_1} = \text{!string.}?double.\oplus \left\{ \begin{array}{l} \text{buy: !string.\& \{accepted: end, rejected: end\},} \\ \text{cancel: end} \end{array} \right\}.$$

The type $\overline{T_1}$ is the behaviour of b inside Buyer.

For the Seller we obtain:

$$\text{Seller} \triangleright \text{req } \Gamma_2, \text{bank: req } T_2, \text{seller: acc } T_1$$

with $T_2 = \text{?double.}?(?string.T_3).\text{!}T_3.T_3$ and $T_3 = \oplus \{ \text{accepted: end, rejected: end} \}.$

For the **Bank** we obtain:

$$\text{Bank} \triangleright \text{req } \Gamma_3, \text{bank}: \text{acc } T_2$$

Interestingly, no type structure is needed for the affine adaptations: cancellation is completely transparent. The variation of **Seller** with an added do-catch, $\text{do req } \overline{\text{bank}} c \text{ catch req } \text{paymate } c$, will simply need $\text{paymate}: \text{req } T_2$ in its interface, i.e., with a type matching that of *bank*, but the original **Seller** can also be typed in the same way by weakening. Similarly, **BuyerMsg** has the same interface as **Buyer**, except that it must include $\text{print}: \text{req string}$, and again the two processes can be assigned the same interface by weakening, if needed. The processes **CheckPriceA**, **CheckPriceB**, and **BuyerCancel** can be assigned the same type for *seller*, namely $\text{req } T_1$, exactly like **Buyer**.

Finally, as we shall see next affinity does not destroy any of the good properties we expect to obtain with session typing.

5 Properties

Typed terms enjoy the expected soundness properties. In particular we have:

Lemma 1 (Substitution). *If $P \triangleright \Gamma$ and $a \notin \text{dom}(\Gamma)$ then $Q\{a/b\} \triangleright \Gamma\{a/b\}$.*

Theorem 1 (Subject Reduction). *If $P \triangleright \Gamma$ and $P \longrightarrow Q$ then $Q \triangleright \Gamma$.*

Proof. The proof relies on several results including Lemma 1. The non-standard case is for cancellations and in particular for $\text{do } \rho \text{ catch } P \mid Q$. However, it can be easily checked that the substitution of ρ by P is sound, because both terms offer the same interface except for the canceled session (and dually P can be thrown away in standard communication).

Theorem 2 (Diamond property). *If $P \triangleright \Gamma$ and $Q_1 \longleftarrow P \longrightarrow Q_2$ then either $Q_1 \equiv Q_2$ or $Q_1 \longrightarrow R \longleftarrow Q_2$.*

Proof. The result is actually easy to establish, since the only critical pairs arise from multiple requests to the same replication or to the same cancellation. However, even in that case the theorem holds because: a) replications are immediately available and functional (*uniform availability*); b) cancellations are persistent. The fact that a_i can never be assigned $\text{req } T$ simplifies the proof.

The above strong confluence property indicates that our sessions are completely deterministic, even considering the possible orderings of requests.

Progress

Our contribution to the theory of session types is *well-behaved affinity*, in the sense that we can guarantee that any session that ends prematurely will not affect the quality of a program. Indeed, if we simply allowed unrestricted weakening,

for example by a type rule $\Gamma \vdash \mathbf{0}$ as done in [13], but without any apparatus at the language level, it would be easy to type a process such as $(\nu a)\bar{a}b.P \mid b(x).Q$ and clearly not only a but also b would be stuck for ever. In this section we prove that this never happens to a well-typed term.

Let us write λ for a prefix ρ that is not a request, i.e., such that $\rho \neq \text{req } \bar{a}b.P$.

Proposition 1. *If $P \triangleright \Gamma$ and $P \equiv (\nu \tilde{a})(H[\lambda] \mid Q)$ and $\text{subject}(\lambda) = b$ and $b \in \text{fn}(Q)$ then $b \in \tilde{a}$.*

This is proved very easily by induction on typing derivations.

We now define a notion of permanently blocked process, which intuitively is a process that cannot proceed in *any* context, either because of deadlock or because of (restricted) sessions without a dual. We will use the fact that linear communications are always under the corresponding bound name, from Proposition 1. As usual $P \not\rightarrow$ means that P cannot reduce.

Definition 1 (Blocked process). *A process P is blocked if $P \not\rightarrow$ and:*

$$P \equiv (\nu \tilde{a} : \tilde{T})(H_1[\rho_1] \mid \cdots \mid H_n[\rho_n]) \quad n \geq 1 \quad \forall i \in \{1, \dots, n\}. \text{subject}(\rho_i) \in \{\tilde{a}\}$$

We give some examples to clarify the definition:

- i) $\text{req } \bar{a}x.\text{req } \bar{b}y.P \mid \text{req } \bar{b}r.\text{req } \bar{a}k.Q$ is not considered blocked since it can reduce properly if we add the appropriate replications.
- ii) $(\nu a)(\text{req } \bar{a}x.\text{req } \bar{b}y.P \mid \text{req } \bar{b}r.\text{req } \bar{a}k.Q)$ is not considered blocked because we can add a replication $\text{acc } b(x).R$ and it will perform one step (before becoming blocked).
- iii) both $(\nu a)(\text{req } \bar{a}x.\text{req } \bar{b}y.P \mid \text{req } \bar{a}k.\text{req } \bar{b}r.Q)$ and $(\nu a, b)(\text{req } \bar{a}x.\text{req } \bar{b}y.P \mid \text{req } \bar{b}r.\text{req } \bar{a}k.Q)$ are blocked, and indeed they have no chance of reducing.
- iv) $a(x).\bar{b}y.P \mid b(z).\bar{a}r.Q$ is not considered blocked, even if it can never reduce, but from Proposition 1 we don't need to consider this case.
- v) $(\nu a, b, c)(a(x).\bar{b}y.P \mid b(z).\bar{c}r.Q \mid c(s).\bar{a}e.R)$ is blocked; there is a cycle spanning all three sub-processes.
- vi) $(\nu a)\bar{a}b.P$ and $(\nu a, b)(\bar{a}b.P \mid b(x).Q)$ are blocked; this is “bad” affinity.
- vii) The above examples can be extended with do-catch, which explains the H -contexts in the definition.

We can now present the main result:

Theorem 3 (Progress). *If $P \triangleright \Gamma$ then:*

- a) *for all Q , $C[\]$ such that $P \equiv C[Q]$, Q is not blocked.*
- b) *if $P \not\rightarrow$, then either $P \equiv \mathbf{0}$ or there exists $Q, \tilde{a}, \Delta, \Theta$ with $Q \triangleright \Delta$ and $Q \not\rightarrow$, such that $(\nu \tilde{a})(P \mid Q) \triangleright \Theta$ and $(\nu \tilde{a})(P \mid Q) \rightarrow$.*

Part a) is shown by Theorem 1 and with the help of a lemma: if $P \triangleright \Gamma$ then P is not blocked. Part b), which is similar to the formulation in [9], is shown by induction on the type derivation.

The theorem is actually very strong, since it holds also for requests, contrary to related works such as [9] where terms enjoy progress only for the linear part

of sessions, i.e., where a request (that may never be activated) can permanently disable any sessions that depend on it.

Moreover, notice that $b)$ in itself is not enough: we could have a blocked subterm in parallel to a request $\text{req } \bar{a}_1 b.P'$, then we could iterate compositions with forwarders $\text{acc } a_i(x).\text{req } \bar{a}_{i+1} x$ and there would always be a reduction. In general, the existence of a “good” Q does not exclude a “bad” one that leads to deadlock. However, from $a)$ we know that there is no subterm that is blocked.

Also, we have not considered circular dependencies for replications; it is easy to check that they cannot lead to deadlock, and actually they cannot be typed.

Finally, we have checked that typed processes are strongly normalising, which is not so surprising since we followed closely the logical principles of Affine Logic. We leave the complete proof of this result, which uses the technique of Reducibility Candidates [12] in conjunction with Theorem 2, to a longer version. Note that Progress (Theorem 3) is in a sense more important, for two reasons: first, a system without progress can still be strongly normalising, since blocked terms are by definition irreducible; second, practical systems typically allow divergence, and in that case the progress property (which we believe can be transferred without surprises to this setting) becomes much more relevant.

6 Related work and future plans

We divide our discussion on the related work in three parts: relaxing linearity in session types, dealing with exceptional behaviour, and logical foundations.

The study of language constructs for exceptional behavior (including exceptional handling and compensation handling) has received significant attention; we refer the reader to a recent overview [10], while concentrating on those works more closely related to ours. Carbone et al. are probably the first to introduce exceptional behaviour in session types [7]. They do so by extending the programming language (the π -calculus) to include a `throw` primitive and a try-catch process. The language of types is also extended with an abstraction for a try-catch block: essentially a pair of types describing the normal and the exceptional behaviour. The extensions allow communication peers to escape, in a *coordinated* manner, from a dialogue and reach another point from where the protocol may progress. Carbone [6] and Capecchi et al. [5] port these ideas to the multi-party setting. Hu et al. present an extension of multi-party session types that allow to specify conversations that may be interrupted [16]. Towards this end, an *interruptible* type constructor is added to the type language, requiring types that govern conversations to be designed with the possible interrupt points in mind. In contrast, we propose a model where programs with and without exceptional behaviour are governed by the same (conventional) types, as it is the norm in functional and object-oriented programming languages.

Caires et al. proposed the conversation calculus [23]. The model introduces the idea of conversation context, providing for a simple mechanism to locally handle exceptional conditions. The language supports error recovery via `throw` and try-catch primitives. No type abstraction is proposed.

Contracts take a different approach by using process-algebra languages [8] or labeled transition systems [3] for describing the communication behaviour of processes. In contrast to session types, where client-service compliance is given by a symmetric duality relation, contracts come equipped with an asymmetric notion of compliance usually requiring that a client and a service reach a successful state. In these works it is possible to end a session (usually on the client side only) prematurely, but there is no mechanism equivalent to our cancellation, no relationship with exception handling, and no clear logical foundations.

Caires and Pfenning gave a Curry-Howard correspondence relating Intuitionistic Linear Logic and session types in a synchronous π -calculus [4]. Although we do not use their types, there is a clear correspondence between $!T_1.T_2$ and $[\![T_1]\!] \otimes [\![T_2]\!]$, and similarly for input. The splitting of the term when composing session endpoints, in our case with (Res), which is standard from [1], was never used in sessions before the work [4]. For output (and request) we followed a different but equivalent approach in which b in $\bar{a}b.P$ is free, when in [4] it would be restricted to appear strictly under P . In fact, we did not change anything compared to the usual output rule [15], which shows that a logical system can be obtained from a standard session system simply by an adaptation of (Res) so that it plays the role of a logical “cut.”

Indeed, the system we presented can be mildly adapted to obtain an embedding of typed processes to proofs of Affine Logic. In any case, our formulation allows to type more processes than Linear Logic interpretations, and to our knowledge it is the first logical account of exceptions in sessions, based on an original interpretation of weakening. Moreover, Propositional Affine Logic is decidable, a result by Kopylov [17], so there are better prospects for type inference.

As part of future work, we would like to develop an algorithmic typing system, along the lines of [22]. We also believe it would be interesting to apply our technique to multiparty sessions [14] based on Proof Nets [20].

Acknowledgments This work was supported by FCT through funding of MULTICORE project, ref. PTDC/EIA-CCO/122547/2010, and LaSIGE Strategic Project, ref. PEst-OE/EEI/UI0408/2014. We would like to thank the anonymous reviewers and also Nobuko Yoshida, Hugo Torres Vieira, Francisco Martins, and the members of the GLOSS group in the University of Lisbon, for their detailed and insightful comments.

References

1. Samson Abramsky. Computational interpretations of linear logic. *Theoretical Computer Science*, 111:3–57, 1993.
2. Andrea Asperti and Luca Roversi. Intuitionistic light affine logic. *ACM Transactions on Computational Logic*, 3(1), 2002.
3. Mario Bravetti and Gianluigi Zavattaro. Contract-based discovery and composition of web services. In *SFM*, volume 5569 of *LNCS*, pages 261–295. Springer, 2009.
4. Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In *CONCUR*, volume 6269 of *LNCS*, pages 222–236. Springer, 2010.

5. Sara Capecchi, Elena Giachino, and Nobuko Yoshida. Global escape in multiparty sessions. In *FSTTCS*, LIPIcs, pages 338–351. Schloss Dagstuhl, 2010.
6. Marco Carbone. Session-based choreography with exceptions. In *PLACES*, volume 241 of *ENTCS*, pages 35–55. Elsevier, 2009.
7. Marco Carbone, Kohei Honda, and Nobuko Yoshida. Structured interactional exceptions in session types. In *CONCUR*, volume 5201 of *LNCS*, pages 402–417. Springer, 2008.
8. Giuseppe Castagna, Nils Gesbert, and Luca Padovani. A theory of contracts for web services. *ACM Transactions on Programming Languages and Systems*, 31(5):1–61, 2009.
9. Mariangiola Dezani-Ciancaglini, Ugo de’ Liguoro, and Nobuko Yoshida. On progress for structured communications. In *TGC’07*, volume 4912 of *LNCS*, pages 257–275. Springer, 2008.
10. Carla Ferreira, Ivan Lanese, António Ravara, Hugo Torres Vieira, and Gianluigi Zavattaro. *Results of the SENSORIA Project 2011*, volume 6582 of *LNCS*, chapter Advanced Mechanisms for Service Combination and Transactions, pages 302–325. Springer, 2011.
11. Simon J. Gay and Malcolm J. Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2/3):191–225, 2005.
12. Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50:1–102, 1987.
13. Marco Giunti. Algorithmic type checking for a pi-calculus with name matching and session types. *The Journal of Logic and Algebraic Programming*, 82(8):263–281, 2013.
14. K. Honda, N. Yoshida, and M. Carbone. Multiparty asynchronous session types. In *POPL*, pages 273–284. ACM, 2008.
15. Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type disciplines for structured communication-based programming. In *ESOP*, volume 1381 of *LNCS*, pages 22–138. Springer, 1998.
16. Raymond Hu, Rumyana Neykova, Nobuko Yoshida, and Romain Demangeon. Practical interruptible conversations: Distributed dynamic verification with session types and Python. In *RV*, volume 8174 of *LNCS*, pages 148–130. Springer, 2013.
17. A.P. Kopylov. Decidability of linear affine logic. *Information and Computation*, 164(1):173 – 198, 2001.
18. Robin Milner. Functions as processes. *Mathematical Structures in Computer Science*, 2(2):119–141, 1992.
19. Robin Milner, Joachim Parrow, and David Walker. A calculus of mobile processes, parts I and II. *Information and Computation*, 100(1), 1992.
20. Dimitris Mostrous. Multiparty sessions based on proof nets. In *Programming Language Approaches to Concurrency and Communication-centric Software (PLACES)*, 2014.
21. Kaku Takeuchi, Kohei Honda, and Makoto Kubo. An interaction-based language and its typing system. In *PARLE ’94*, pages 398–413. Springer, 1994.
22. Vasco T. Vasconcelos. Fundamentals of session types. *Information and Computation*, 217:52–70, 2012.
23. Hugo T. Vieira, Luís Caires, and João C. Seco. The conversation calculus: A model of service-oriented computation. In *ESOP*, volume 4960 of *LNCS*, pages 269–283. Springer, 2008.